Tipos Abstratos de Dados

Prof. Henrique Batista da Silva

Abstração

- Processo de representar um grupo de entidades através de seus atributos comuns.

- Feita a abstração, cada entidade particular (instância) do grupo é considerada somente pelos seus atributos particulares.

Encapsulamento

- Encapsulamento é um agrupamento de subprogramas + dados que é compilado separada/independentemente.
- Um encapsulamento é portanto um sistema abstraído.
- Muitas vezes os encapsulamentos são colocados em bibliotecas.

Encapsulamento

- Encapsulamento pode ser definido como a tarefa de tornar um objeto o mais auto suficiente possível.

- *Para se dirigir um carro não é necessário entender como que o motor do carro funciona.*

- *Para se utilizar o computador, não é necessário saber como que o sistema operacional se comunica com o hardware da máquina para diversas funções.*

Abstração x Encapsulamento

- A abstração define as entidade que representa um objeto, e o encapsulamento esconde detalhes daquele objeto, ocultando detalhes de seu funcionamento.

Objetos

- Um tipo abstrato de dados é um encapsulamento que inclui somente um tipo específico de dado e os subprogramas que fornecem as operações para este tipo

- Detalhes de implementação do tipo ficam ocultos das unidades fora do encapsulamento que o contém

Objetos

- Um objeto é uma variável (instância) de um tipo abstrato de dados, declarada por alguma unidade

- Programação orientada a objetos consiste no uso de objetos no desenvolvimento do software

Classes

- Uma Classe é um tipo de dados semelhante a uma estrutura, constituídas por Atributos e Métodos.
- Atributos de classe são semelhantes a campos de estruturas;
- Por exemplo: uma classe Data poderia conter os atributos *dia, mes* e *ano*.
- Métodos de classe são funções que operam sobre os atributos.

Classes

- Se declararmos uma variável cujo tipo é uma classe, esta variável será composta, assim como nas estruturas.
- Haverá uma cópia de cada um dos atributos para cada variável declarada daquela classe.

Classe x Objeto

- Uma classe é um modelo para algo que deve ser criado.
- Por exemplo, uma “Receita para Bolo de Chocolate”, será usada como um modelo para o bolo que será criado.
- De forma análoga, a receita é uma classe, e o bolo é um objeto.

Classe x Objeto

- Uma classe não é utilizada diretamente.
- A classe é utilizada somente para criar objetos baseados nela, e são os objetos que serão realmente utilizados.
- Por definição, um objeto é a instância de uma classe.

Classe x Objeto

- Exemplo de classe e sua instância (objeto da classe):

```
//Classe Produto
public class Produto
{
        ...
}

//instância da Classe Produto
Produto meuProduto = new Produto();
```

Classes

- Exemplo: se “Data” for uma classe definida no programa, então a declaração:

  Data d1, d2, d3;

- criará três variáveis cujo tipo é Data;
- cada uma destas variáveis possui os campos *dia*, *mes* e *ano*.

Classe Publica e Privada

- Uma classe é geralmente dividida em duas partes:
  - a parte privativa contém elementos que serão de uso exclusivo da classe.
  - a parte pública contém os elementos exportados pela classe.
- Por simplicidade, todos os atributos são privativos e todos os métodos são públicos.

Construtores

- Uma classe pode possuir também métodos especiais denominados construtores.
- Um construtor é um método especial que possui o mesmo nome da classe e não possui tipo de retorno, tendo por finalidade iniciar os atributos do objeto.

Construtores

```
class Data {
        private: int dia, mes, ano;

        public: Data(int d, int m, int a)        {
                dia = d; mes = m; ano = a;
        }

        bool bissexto()
        {
                if (ano % 4 != 0) return false;
                else if (ano % 100 == 0 && ano % 400 != 0) return false;
                else return true;
        }

        void imprime()
        {
                Console.WriteLine(dia + "/" + mes + "/" + ano);
        }
};
```

Construtores

```
void main()
{
        Data d(10,3,2003);

        d.imprime();
}
```

```
void main()
{
        Data d1(10,3,2003),  d2(3,7,2002);

        d1.imprime();
        d2.imprime();
}
```

Construtores

- Cada objeto possui uma cópia de cada um dos atributos de modo que a chamada d1.imprime() imprimirá os atributos de d1 e a chamada d2.imprime() imprimirá os atributos de d2.

Construtor default

- Construtor default, que é um construtor sem parâmetros.
- É um método sem parâmetros e sem tipo de retorno e cujo nome é o mesmo da classe.
- Este construtor especial serve para iniciar objetos com valores padrão.

Construtor default

```
class Data {
        private: int dia, mes, ano;
        public: Data(int d, int m, int a)        {
                dia = d; mes = m; ano = a;
        }
        Data() // construtor default
        {
                dia = 1; mes = 1; ano = 2003;
        }
        bool bissexto()
        {
                if (ano % 4 != 0) return false;
                else if (ano % 100 == 0 && ano % 400 != 0) return false;
                else return true;
        }
        void imprime()
        {
                Console.WriteLine(dia + "/" + mes + "/" + ano);
        }
};
```

Tipos Abstratos de Dados

- Tipos simples de dados são grupos de valores indivisíveis.
  - *int, bool, char, double, etc.*

- Tipo abstrato de dados (TAD) pode ser visto como um modelo matemático, acompanhado de operações definidas sobre o modelo.

- Um TAD pode ser usado para encapsular tipos simples de dados.

Tipos Abstratos de Dados

- É um conjunto de dados munido de operações sobre estes dados.

- Por exemplo, o TAD Data representa todas as datas existentes, munidas das operações usuais sobre datas.

Tipos Abstratos de Dados

- Na Programação Orientada por Objetos, dizemos que uma Classe implementa um Tipo Abstrato de Dados.

- Uma classe contém os elementos que representam o tipo, e agrupa todas as operações realizadas sobre estes elementos.

Tipos Abstratos de Dados

- Por exemplo, considere uma aplicação que utiliza uma lista de inteiros:
  - Definição da TAD: Lista
  - Operações da TAD Lista:
    - Criar a lista vazia;
    - Obter o primeiro elemento da lista;
    - Inserir um elemento na lista.
  - A lista pode ser implementada utilizando uma estrutura de dados como um arranjo.

Tipos Abstratos de Dados

- Cada operação (método) do TAD Lista pode ser implementada através de procedimentos ou funções.

- Se existir a necessidade de alteração do TAD, esta é realizada na parte encapsulado, sem afetar outras partes do código.

Tipos Abstratos de Dados

- Na implementação de um TAD, alguns fatores elevam a qualidade do código:
  - Ocultamento de informações: é importante que o cliente conheça o menor número possível de informações a respeito do TAD.
  - Interface simples e bem definida: o TAD deve ser definido de forma que seja fácil utilizar as suas funcionalidades (alterar o algoritmo sem alterar a assinatura).

Referências

- CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

- DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).

- ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p. ISBN 8522105251 (Disponível no Acervo).

- Microsoft: msdn.microsoft.com